Phase 2: Transformer 架构各组件详解

1. 整体架构概览

本作业实现的是 **decoder-only, pre-norm Transformer LM**，与 GPT-2/LLaMA 同族。

```
输入: token IDs x ∈ Z^(B × T)     (B=batch_size, T=seq_len)
                 │
                 ▼
         Token Embedding           x → E[x] ∈ R^(B × T × D)
                 │
                 ▼
         Transformer Block × L     (L = num_layers)
         ┌──────────────────┐
         │  RMSNorm → MHA   │──── + residual
         │  RMSNorm → FFN   │──── + residual
         └──────────────────┘
                 │
                 ▼
         Final RMSNorm             ∈ R^(B × T × D)
                 │
                 ▼
         LM Head (Linear)          → logits ∈ R^(B × T × V)  (V=vocab_size)
```

维度约定：

- B = batch_size
- T = sequence_length (≤ context_length)
- D = d_model
- H = num_heads
- d_k = d_v = D / H（每个 head 的维度）
- d_ff ≈ 8/3 × D，取 64 的整数倍

2. 基础构件

2.1 Linear 层

数学：y = Wx （无 bias）

张量形状：

- 权重 W ∈ R^(d_out × d_in)，存储为 `nn.Parameter`
- 输入 x： `(..., d_in)` → 输出 y： `(..., d_out)`
- `...` 表示任意数量的 batch 维度

forward 实现思路：

- 最直接： `x @ W.T` （矩阵乘法，PyTorch 自动处理 batch 维度广播）
- 或用 einsum： `einsum('...i, oi -> ...o', x, W)`

初始化：截断正态 N(0, σ^2)，σ^2 = 2/(d_in + d_out)，截断到 [-3σ, 3σ]

2.2 Embedding 层

数学：给定 token ID i，返回 embedding 矩阵第 i 行

张量形状：

- embedding 矩阵 E ∈ R^(vocab_size × d_model)
- 输入 token_ids： `(...)` 任意形状的整数 → 输出 `(..., d_model)`

forward 实现思路：

- 直接索引： `self.weight[token_ids]`
- PyTorch 的高级索引会自动处理任意形状

初始化：截断正态 N(0, 1)，截断到 [-3, 3]

3. RMSNorm

3.1 与 LayerNorm 的对比

特性	LayerNorm	RMSNorm
减均值	是	否
除以标准差	是	否（除以 RMS）
可学参数	gain g 和 bias b	只有 gain g
计算量	两次 reduction	一次 reduction

3.2 数学定义

```
RMS(a) = sqrt( (1/d_model) * Σ a_i² + ε )

RMSNorm(a)_i = (a_i / RMS(a)) * g_i
```

- a ∈ R^d_model 是输入向量
- g ∈ R^d_model 是可学习的缩放参数（初始化为全 1）
- ε 是数值稳定项（默认 1e-5）

3.3 实现要点

dtype 注意事项：

- 输入可能是 float16 或 bfloat16
- 平方运算在低精度下容易溢出
- 正确做法：`输入 upcast 到 float32 → 计算 RMS → 归一化 → cast 回原 dtype`

形状处理：

- 输入 x：`(..., d_model)`，其中 `...` 是任意 batch 维度
- RMS 沿最后一维计算
- g 广播到所有 batch 维度

计算流程：

```
x_float = x.to(float32)
rms = sqrt(mean(x_float², dim=-1, keepdim=True) + eps)
x_normed = x_float / rms
output = (x_normed * g).to(x.dtype)
```

4. SiLU 与 SwiGLU

4.1 SiLU（Sigmoid Linear Unit）

```
SiLU(x) = x · σ(x) = x / (1 + e^(-x))
```

- 也叫 Swish 激活函数
- 平滑非单调：在 x ≈ -1.28 处有一个浅谷
- 实现时直接用 `x * torch.sigmoid(x)` 即可

4.2 GLU（Gated Linear Unit）机制

GLU 的核心思想：用一个"门"来控制信息流动

```
GLU(x) = σ(W₁x) ⊙ W₃x
         ^^^^^^^^   ^^^^
         门控信号    信息流
```

- ⊙ 表示逐元素乘法（Hadamard product）
- 门控信号通过 sigmoid 压缩到 [0,1]，决定每个维度"放行"多少信息

4.3 SwiGLU = SiLU + GLU

```
FFN(x) = W₂ · (SiLU(W₁x) ⊙ W₃x)
```

三个权重矩阵的角色：

- $W_1 \in R^{(d_ff \times d_model)}$：将输入投影到 d_ff 维，经 SiLU 后作为门控
- $W_3 \in R^{(d_ff \times d_model)}$：将输入投影到 d_ff 维，作为信息通道
- $W_2 \in R^{(d_model \times d_ff)}$：将结果投影回 d_model 维

为什么 d_ff ≈ 8/3 × d_model：

- 传统 FFN（ReLU）用 d_ff = 4 × d_model，有 2 个矩阵，参数量 = 2 × d_model × 4d_model = $8d^2$
- SwiGLU 有 3 个矩阵，为了保持总参数量相当：3 × d_model × d_ff ≈ $8d^2$，所以 d_ff ≈ 8/3 × d_model
- 取 64 的倍数是为了 GPU 对齐（tensor core 通常以 8/16/64 为单位计算）

数据流形状：

```
x: (..., d_model)
    │
    ├── W₁x: (..., d_ff)  → SiLU → gate: (..., d_ff)
    │
    ├── W₃x: (..., d_ff)  → value: (..., d_ff)
    │
    └── gate ⊙ value: (..., d_ff)
          │
          └── W₂(gate ⊙ value): (..., d_model)
```

5. Scaled Dot-Product Attention

5.1 数学公式

```
Attention(Q, K, V) = softmax(QK^T / √d_k) · V
```

5.2 逐步分解

给定：

- Q ∈ R^(..., n, d_k) — n 个 query 向量
- K ∈ R^(..., m, d_k) — m 个 key 向量
- V ∈ R^(..., m, d_v) — m 个 value 向量

```
Step 1: 计算注意力分数
  scores = Q @ K^T / √d_k     ∈ R^(..., n, m)

  为什么除以 √d_k?
  - Q 和 K 的每个元素大约是 N(0,1)
  - 点积 = Σ q_i·k_i，方差 ≈ d_k
  - 除以 √d_k 使方差回到 1
  - 否则 d_k 很大时，softmax 输入过大，梯度趋近于 0

Step 2: 应用 mask（如果有）
  if mask is not None:
    scores[mask == False] = -inf    # 或一个很大的负数

Step 3: Softmax
  weights = softmax(scores, dim=-1)   ∈ R^(..., n, m)
  每行和为 1（或为 0，如果整行都被 mask）

Step 4: 加权求和
  output = weights @ V              ∈ R^(..., n, d_v)
```

5.3 因果 Mask（Causal Mask）

自回归语言模型中，位置 i 只能看到位置 ≤ i 的 token：

```
mask[i][j] = True  if j ≤ i
           = False if j > i

对于 seq_len=4 的 mask:
  T . . .
  T T . .
  T T T .
  T T T T

构造方式:
  mask = torch.tril(torch.ones(T, T, dtype=torch.bool))
  或
  mask = torch.ones(T, T, dtype=torch.bool).triu(diagonal=1).logical_not()
```

5.4 Softmax 的数值稳定性

```
softmax(v)_i = exp(v_i) / Σ_j exp(v_j)
```

问题： `exp(v_i)` 当 v_i 很大时会溢出。

解决：减去最大值

```
v_max = max(v)
softmax(v)_i = exp(v_i - v_max) / Σ_j exp(v_j - v_max)
```

这不改变结果（分子分母同乘 exp(-v_max) 约掉），但保证 exp 的输入 ≤ 0。

6. Rotary Position Embeddings (RoPE)

6.1 动机

Transformer 本身是 permutation invariant 的——如果不加位置信息，交换输入 token 顺序不会改变输出。

位置编码的方式：

- **绝对位置编码**（如原始 Transformer）：给每个位置加一个固定/可学习向量
- **相对位置编码**（如 RoPE）：让注意力分数自然地依赖于 token 之间的相对距离

6.2 核心思想

RoPE 把 d_k 维向量视为 d_k/2 个 2D 平面上的点，对每个平面施加不同频率的旋转。

对于位置 pos、第 i 对维度（i = 0, 1, ..., d_k/2 - 1）：

```
旋转角度 θ_i(pos) = pos / (base^(2i/d_k))

其中 base = 10000（默认值，即 theta 参数）
```

旋转操作：

```
[x_{2i}  ]     [cos θ   -sin θ] [x_{2i}  ]
[x_{2i+1}]  =  [sin θ    cos θ] [x_{2i+1}]
```

6.3 为什么旋转能编码相对位置

设 q 在位置 p_q，k 在位置 p_k。

旋转后的点积：

```
<R(p_q)·q, R(p_k)·k> = <R(p_q - p_k)·q, k>
```

即：旋转后的 q·k 只依赖于 **p_q - p_k**（相对位置差），这正是我们想要的！

6.4 频率设计的直觉

```
θ_0 = pos / base^0          = pos       （高频，变化快）
θ_1 = pos / base^(2/d_k)                 （稍低频）
...
θ_{d_k/2-1} = pos / base^1  = pos/base  （低频，变化慢）
```

- 前面的维度对：频率高，捕捉近距离的位置差异
- 后面的维度对：频率低，捕捉远距离的位置关系
- 类似正弦位置编码的"多频率"设计

6.5 实现方式

有两种常见的实现方式：

方式 A：显式旋转矩阵

```
对每对 (x_{2i}, x_{2i+1}):
  x_{2i}'   = x_{2i} * cos(θ) - x_{2i+1} * sin(θ)
  x_{2i+1}' = x_{2i} * sin(θ) + x_{2i+1} * cos(θ)
```

方式 B：复数视角

```
把 (x_{2i}, x_{2i+1}) 看作复数 x_{2i} + j·x_{2i+1}
旋转 = 乘以 e^(jθ) = cos(θ) + j·sin(θ)
```

预计算与缓存：

- 在 `__init__` 中预计算所有 (position, dimension_pair) 组合的 cos 和 sin 值
- 用 `self.register_buffer("cos_cached", ..., persistent=False)` 存储
- `persistent=False` 表示不会被 `state_dict()` 保存（因为可以重新计算）

形状处理：

```
输入 x: (..., seq_len, d_k)
token_positions: (..., seq_len)

预计算:
  freqs: (max_seq_len, d_k/2)
    freqs[pos][i] = pos / base^(2i/d_k)
  cos_cached: (max_seq_len, d_k/2) = cos(freqs)
  sin_cached: (max_seq_len, d_k/2) = sin(freqs)

forward:
  用 token_positions 从 cos_cached/sin_cached 索引出当前 batch 需要的值
  对 x 的偶数和奇数维度分别应用旋转
```

6.6 为什么只旋转 Q 和 K

- Q·K 的点积决定了"哪些位置应该关注哪些位置"——这需要位置信息
- V 是被聚合的信息本身——不需要位置偏置
- 旋转 Q 和 K 后，注意力分数自然包含了相对位置信息
- V 保持原样，被按注意力权重加权求和

7. Multi-Head Self-Attention (MHA)

7.1 为什么要"多头"

单头注意力：所有信息压缩到一组 Q, K, V 中。
多头注意力：让不同 head 关注不同类型的特征（语法关系、语义关系、局部/全局关系等）。

7.2 数学定义

```
Q = W_Q · x    ∈ R^(B, T, D)    — 然后 reshape 为 (B, H, T, d_k)
K = W_K · x    ∈ R^(B, T, D)    — 然后 reshape 为 (B, H, T, d_k)
V = W_V · x    ∈ R^(B, T, D)    — 然后 reshape 为 (B, H, T, d_v)

其中 d_k = d_v = D / H

对每个 head h:
  head_h = Attention(Q[:, h], K[:, h], V[:, h])   ∈ R^(B, T, d_v)

output = Concat(head_0, ..., head_{H-1}) @ W_O
       = reshape(all_heads, (B, T, D)) @ W_O      ∈ R^(B, T, D)
```

7.3 "一次矩阵乘法处理所有 head"的技巧

朴素做法：每个 head 有独立的 W_Q^h ∈ R^(d_k × D)，做 H 次矩阵乘法。

高效做法：把所有 head 的投影矩阵堆叠成一个大矩阵：

```
W_Q = stack([W_Q^0, W_Q^1, ..., W_Q^{H-1}])  ∈ R^(H·d_k × D) = R^(D × D)

Q_all = x @ W_Q^T                              ∈ R^(B, T, D)
Q_all = Q_all.reshape(B, T, H, d_k)
Q_all = Q_all.transpose(1, 2)                  ∈ R^(B, H, T, d_k)
```

这样只需一次矩阵乘法 + reshape + transpose。

7.4 完整的 MHA forward 流程

```
输入 x: (B, T, D)

1. 线性投影 (一次矩阵乘法):
   Q = Linear_Q(x)    → (B, T, D)
   K = Linear_K(x)    → (B, T, D)
   V = Linear_V(x)    → (B, T, D)

2. reshape 为多头:
   Q → (B, T, H, d_k) → transpose → (B, H, T, d_k)
   K → (B, T, H, d_k) → transpose → (B, H, T, d_k)
   V → (B, T, H, d_v) → transpose → (B, H, T, d_v)

3. 对 Q 和 K 应用 RoPE:
   Q = RoPE(Q, positions)
   K = RoPE(K, positions)

4. Scaled dot-product attention (带因果 mask):
   attn_output = Attention(Q, K, V, causal_mask)  → (B, H, T, d_v)

5. 拼接 head:
   attn_output → transpose → (B, T, H, d_v) → reshape → (B, T, D)

6. 输出投影:
   output = Linear_O(attn_output)              → (B, T, D)
```

7.5 token_positions 的作用

- 通常在训练时，positions 就是 `[0, 1, 2, ..., T-1]`
- 但在推理/生成时，如果使用 KV-cache，新 token 的 position 不是 0 而是它在完整序列中的位置
- 因此 RoPE 需要接收显式的 position 张量，而非假设从 0 开始

8. Pre-norm Transformer Block

8.1 结构

```
def forward(self, x):
    # 子层 1: Self-Attention
    x = x + MHA(RMSNorm_1(x))

    # 子层 2: Feed-Forward
    x = x + FFN(RMSNorm_2(x))

    return x
```

8.2 Pre-norm vs Post-norm

```
Pre-norm:   x = x + Sublayer(Norm(x))     ← 本作业使用
Post-norm:  x = Norm(x + Sublayer(x))     ← 原始 Transformer
```

Pre-norm 的优势：

- Residual stream 更"干净"——norm 只在 sublayer 内部，不影响主通路
- 梯度可以直接通过 residual connection 流回，不经过 norm
- 对初始化和学习率更鲁棒，训练更稳定
- 代价：可能需要在最后一个 block 后额外加一个 norm（即 `ln_final`）

8.3 Residual Connection 的直觉

```
x_{l+1} = x_l + F(x_l)
```

- 梯度回传时：∂x_{l+1}/∂x_l = I + ∂F/∂x_l
- 即使 ∂F/∂x_l 很小，梯度仍然有 I（恒等）分量
- 这大大缓解了深层网络的梯度消失问题

9. 完整 Transformer LM

9.1 组装

```
class TransformerLM(nn.Module):
    # 包含:
    # - token_embeddings: Embedding(vocab_size, d_model)
    # - layers: ModuleList of TransformerBlock × num_layers
    # - ln_final: RMSNorm(d_model)
    # - lm_head: Linear(d_model, vocab_size)  无 bias

    def forward(self, token_ids):
        # token_ids: (B, T) 整数

        x = token_embeddings(token_ids)     # (B, T, D)

        for layer in layers:
            x = layer(x)                     # (B, T, D)

        x = ln_final(x)                     # (B, T, D)
        logits = lm_head(x)                  # (B, T, V)

        return logits
```

9.2 输出的含义

- `logits[b, t, :]` 是 batch 中第 b 个样本、第 t 个位置预测的下一个 token 的未归一化 log 概率
- 训练时用 cross-entropy： `loss = CE(logits[b, t, :], target[b, t])`
- 生成时取最后一个位置的 logits 做 softmax，采样下一个 token

9.3 参数量估算（FLOPs 题的基础）

组件	参数量
Token Embedding	V × D
每层 Q/K/V proj	3 × D × D
每层 O proj	D × D
每层 FFN (W1,W2,W3)	3 × D × d_ff

组件	参数量
每层 RMSNorm × 2	2 × D
Final RMSNorm	D
LM Head	V × D
总计	**$2VD + L(4D^2 + 3D \cdot d_ff + 2D) + D$**

对 GPT-2 XL (D=1600, L=48, d_ff=6400, V=50257):

- Embedding + LM Head: ≈ 161M
- 每层 attention: $4 \times 1600^2 = 10.24M$
- 每层 FFN: 3 × 1600 × 6400 = 30.72M
- 每层小计: ≈ 41M
- 48 层: ≈ 1968M
- 总计: ≈ 2.1B 参数（~8.4GB float32）